

Разработка приложения «Анализ финансов»

Описание проекта

В современном мире финансовых технологий объём транзакционных данных каждый день стремительно растёт. Компании, работающие в этой сфере, должны эффективно обрабатывать и анализировать такие данные, чтобы оставаться конкурентоспособными, обеспечивать безопасность и предлагать клиентам современные сервисы.

Мы, как одна из ведущих компаний в этой области, стремимся улучшить пользовательский опыт. Наша цель — обеспечить быстрый и надёжный сервис, включая:

- мгновенные переводы;
- мониторинг транзакций в режиме реального времени;
- предотвращение мошенничества.

Мы сталкиваемся с большим потоком транзакционных данных, который нужно не только хранить, но и анализировать. Это важно:

- для выявления аномалий;
- создания отчётов для внутренних и внешних заинтересованных сторон;
- оптимизации потоков данных для ускорения бизнес-процессов.

Для решения этой проблемы мы хотим разработать аналитическое приложение, которое будет:

- безопасно и быстро импортировать данные транзакций из разных источников;
- анализировать данные: искать тренды, выявлять аномалии и формировать статистику;
- предоставлять ясные и точные отчёты для финансовых аналитиков и управляющего персонала.

Ваша задача — разработать прототип решения, которое будет:

- читать и обрабатывать данные из файлов;
- применять алгоритмы анализа данных;
- генерировать отчёты в соответствии с предусмотренными стандартами качества.

Этот проект поможет улучшить клиентский сервис и даст компании долгосрочные преимущества в конкурентной борьбе на финансовом рынке.

Техническое задание на разработку

Описание поведения

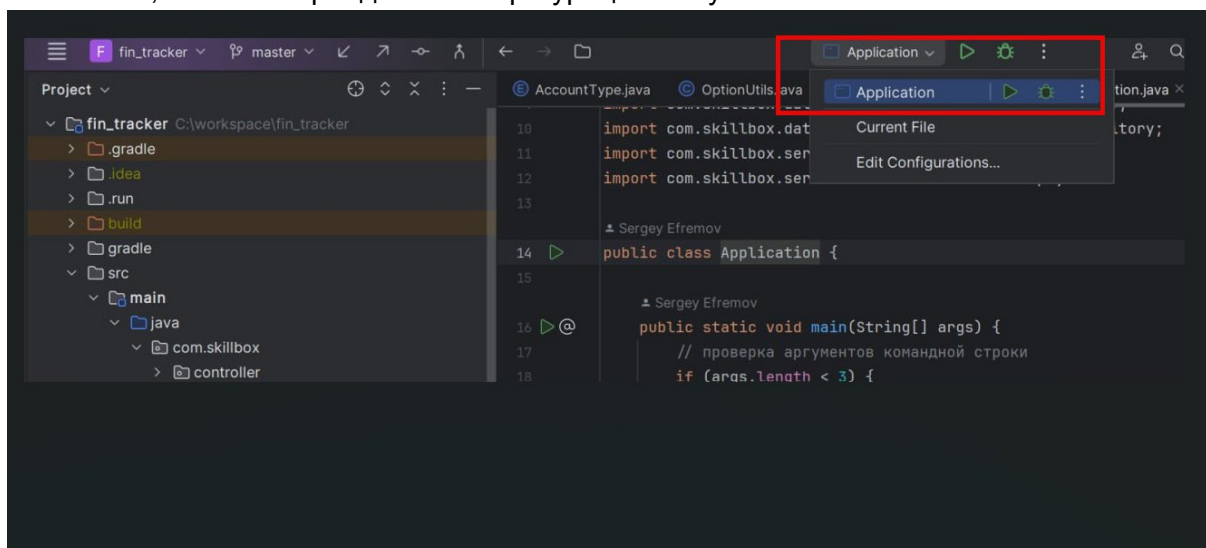
Программа «Анализ финансов» предназначена для обработки и анализа транзакционных данных, полученных из файлов. С её помощью пользователь может гибко настраивать параметры поиска, группировки и агрегации данных, чтобы получать детализированную финансовую аналитику. Ниже приведено описание работы программы и взаимодействие с пользователем.

Начало работы и запуск проекта

Перед началом работы вы получите доступ к шаблону проекта. Проект использует систему сборки Gradle, поэтому в корне проекта вы найдёте файл `build.gradle` с инструкциями по сборке для Gradle. Пока в этом файле вам достаточно лишь проверить версию Java и при необходимости заменить её на ту, которая установлена на вашем компьютере.

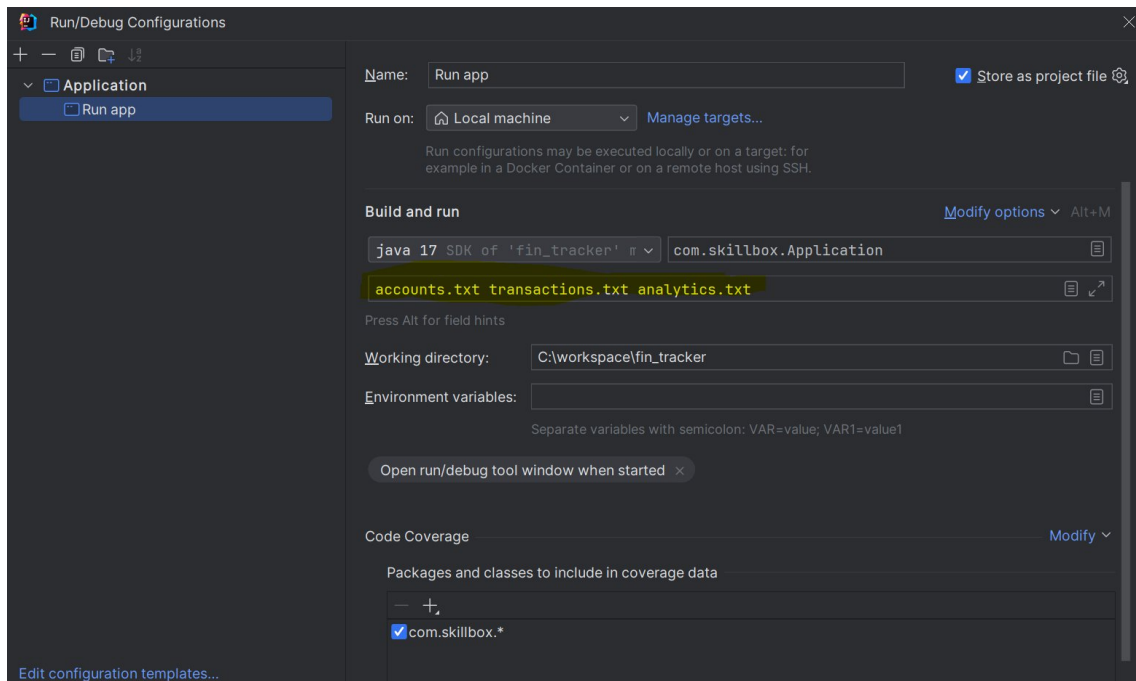
Запустить проект позволит класс `Application` в пакете `com.skillbox`. При первом запуске работа программы может завершиться ошибкой, так как ожидается, что на вход приложение получит определённые параметры.

Чтобы отредактировать эти параметры, запустите приложение, дождитесь сообщения об ошибке, а затем перейдите в конфигурацию запуска:



Здесь и далее — скриншоты IntelliJ IDEA / JetBrains

Чтобы изменить конфигурацию, выберите пункт **Edit Configuration**. Так вы откроете окно, где можно изменить параметры запуска:



На вход программе следует передать три параметра:

- имя входного файла с информацией о счетах;
- имя входного файла с данными о транзакциях;
- имя файла для сохранения результатов анализа.

Эти параметры можно ввести в окне конфигурации: на рисунке они подсвечены жёлтым. Имена разделены пробелами. Входные файлы должны существовать и лежать в папке с проектом.

В итоге эти имена будут доступны в методе `main(String[] args)` главного класса:

```
public static void main(String[] args) {
    // проверка аргументов командной строки
    if (args.length < 3) {
        throw new IllegalArgumentException("Необходимо указать имена
файлов для входных данных аккаунтов и транзакций, а также для выходного
файла.");
    }
    // имя входного файла с информацией об аккаунтах
    String accountFilename = args[0];
    // имя входного файла с информацией о транзакциях
    String transactionFilename = args[1];
    // имя выходного файла для записи результата
    String outputFilename = args[2];
```

Если параметров меньше трёх, запуск проекта закончится ошибкой. В шаблоне проекта приложены примеры входных файлов:

- `accounts.txt` — данные о счетах;
- `transactions.txt` — данные о транзакциях.

Вы можете в любой момент добавить свои файлы (тогда нужно внести изменения в конфигурации) или изменить существующие.

Формат входных файлов

На вход программы подаются два файла: с информацией о счетах (аккаунтах) и о транзакциях. Рассмотрим формат и примеры этих файлов.

Счета (аккаунты)

Информация о каждом счёте хранится в одной строке. Каждое поле разделено запятой, например так:

```
345,0,10123 #первый счёт
61823,2,20234
48922,0,30345
1534,1,60678
61234,2,49567
```

Информация о счёте состоит из трёх полей:

- первое поле — ID счёта;
- второе поле — тип счёта, где 0 — текущий, 1 — сберегательный, 2 — кредитный счёт;
- третье поле — ID пользователя.

У каждого пользователя может быть несколько типов счетов, поэтому в файле могут встречаться одинаковые ID пользователя, но ID счёта всегда будет уникальным.

Транзакции

Информация о каждой транзакции хранится в отдельной строке, поля транзакции разделены запятой. Например, файл с транзакциями выглядит так:

```
1534,6,2020-03-22T19:00:00,Entertainment,-45.00,Recurrent,monthly;7
61234,8,2020-04-15T14:15:00,Salary,5000.00,Taxable,0.20
48922,10,2020-05-10T11:00:00,Utilities,-100.00,Taxable,0.10
61234,12,2020-06-18T16:20:00,Leisure,-75.99,ForeignCurrency,0.85
23459,14,2020-07-05T07:45:00,Food,-12.50,Regular,
```

Расшифруем информацию о транзакции.

1. ID счёта могут повторяться, так как пользователи могут неоднократно совершать транзакции с одного счёта.
2. ID транзакции — уникальное числовое поле, характеризующее конкретную транзакцию.
3. Дата и время транзакции хранятся в формате ISO: дата в формате `yyyy-mm-dd`, затем разделитель — символ `T` и время в формате `hh:mm:ss`. В классе `DateTimeFormatter` есть специальная константа для этого формата — `ISO_LOCAL_DATE_TIME`.

4. Категория транзакции — произвольное текстовое поле.
5. Сумма транзакции может быть положительной для поступлений средств на счета клиента и отрицательной для трат со счёта.
6. Тип транзакции — текстовое поле, характеризующее транзакцию. Может принимать следующие значения:
 - `Regular` — обычная транзакция;
 - `Taxable` — транзакция, сумма которой облагается налогом;
 - `Recurrent` — транзакция, повторяющаяся с определённой периодичностью;
 - `ForeignCurrency` — транзакция в иностранной валюте;
 - `Commentable` — транзакция с дополнительной информацией, комментарием.
7. Далее следуют поля: их тип, состав и значение специфичны для каждого типа транзакции. Таких полей может быть несколько, в этом случае они разделяются точкой с запятой. Перечислим их значения и состав для каждого типа транзакции.

`Regular`: дополнительные поля отсутствуют.

`Taxable` — одно поле: число, обозначающее процентную ставку налога. Необходимо рассчитать налог и вычесть его из исходной суммы транзакции.

`Recurrent` содержит два поля, разделённых точкой с запятой:

- паттерн, который обозначает периодичность повторения транзакции и может принимать такие значения:
 - `hourly` — транзакцию следует повторять каждый час начиная с даты или времени совершения транзакции;
 - `daily` — транзакцию следует повторять ежедневно;
 - `weekly` — транзакцию следует повторять еженедельно;
 - `biweekly` — транзакцию следует повторять каждые две недели (раз в 14 дней);
 - `monthly` — транзакция повторяется каждый месяц;
 - `yearly` — транзакция повторяется ежегодно.
- количество повторений транзакции.

`ForeignCurrency` — курс иностранной валюты по отношению к базовой валюте, в которой считаются все транзакции. Сумма такой транзакции указана в иностранной валюте, эту сумму необходимо привести к базовой, умножив на курс.

`Commentable` — комментарии к транзакции. Их может быть несколько, в таком случае они разделяются точкой с запятой.

Главное меню

После запуска и проверки наличия входных параметров программа демонстрирует главное меню и предлагает пользователю выбрать нужную операцию:

Анализ финансов

- 0 — выход из приложения;
- 1 — задать критерии поиска транзакций;
- 2 — выбрать поле для группировки;
- 3 — выбрать функцию агрегации;
- 4 — рассчитать и вывести аналитику;
- 5 — сохранить аналитику.

Введите нужную опцию и нажмите Enter.

Пользователь должен:

- задать критерии фильтрации (поиска) транзакций;
- выбрать поле группировки транзакций и функцию агрегации.

После этого можно рассчитать аналитические данные и при необходимости сохранить результат расчёта. Без критериев фильтрации аналитика считается по всем доступным транзакциям. Можно пропустить опцию группировки, тогда она не будет осуществляться. Также можно не выбирать функцию агрегации, в этом случае используется функция суммирования.

Критерии поиска транзакций

В этом меню можно выбрать критерии для поиска и фильтрации транзакций:

Выберите способ поиска транзакции:

- 0 — сохранить поиск и вернуться назад;
- 1 — выбрать все транзакции (сбросит все заданные ранее фильтры);
- 2 — искать по категориям;
- 3 — искать по диапазону дат;
- 4 — искать по диапазону суммы транзакций;
- 5 — искать по комментарию (для транзакций, поддерживающих комментарии).

Введите нужную опцию и нажмите Enter.

Пользователь может выбирать пункты меню от второго до пятого в любом порядке и задавать критерии фильтрации. Эти фильтры объединяются друг с другом с помощью логического оператора «И»: в конечную выборку транзакций войдут только записи, которые удовлетворяют всем выбранным условиям одновременно. Например, если пользователь выберет фильтрацию по категории «Покупки» и задаст диапазон дат «с 01.01.2023 по 31.01.2023», программа вернёт только транзакции, соответствующие и выбранной категории, и указанному диапазону дат.

В опции 0 выбранные фильтры сохраняются, а пользователь возвращается в главное меню. В опции 1 выбранные ранее фильтры сбрасываются, и пользователь также возвращается в главное меню.

Рассмотрим примеры ввода пользователя для остальных пунктов.

Поиск по категориям

После выбора опции 2 пользователь может ввести одну категорию:

Введите категорию (Enter — поиск по всем категориям): Health

Если нажать Enter или ввести строку только из пробелов, фильтрации по категории не будет. А если пользователь введёт название категории и нажмёт Enter, проанализируются только транзакции с выбранной категорией. Регистр символов при этом учитывается.

Поиск по диапазону дат

После выбора опции 3 пользователь может установить начальную и конечную даты:

Будут найдены транзакции, которые находятся в диапазоне дат, а также повторяющиеся транзакции, которые выполнятся в указанном диапазоне. Формат даты: ГГГГ-ММ-ДД, например 2024-09-27.

Введите начальную дату транзакции (Enter — не ограничивать начальную дату): 2023-01-01

Введите конечную дату транзакции (Enter — не ограничивать конечную дату).

Пользователь может ввести дату в указанном формате или оставить ввод пустым, нажав Enter. В последнем случае даты не будут учитываться в поиске.

Важно: повторяющиеся транзакции (типа `Recurrent`) должны попадать в результаты поиска и в том случае, если будут выполнены в указанный диапазон дат.

В качестве примера возьмём повторяющуюся транзакцию от 1 января 2020 года. Она повторяется ежегодно и диапазон её действия — 10 лет начиная с 31 декабря 2015 года. Если пользователь вводит диапазон дат, например «от 31/12/2024 до 2/1/2025», эта транзакция должна быть учтена в аналитике с таким фильтром.

Поиск по диапазону суммы транзакций

При выборе опции 4 пользователь может установить диапазон суммы транзакции, которые попадут в фильтр:

Введите минимальную сумму транзакции (Enter — не ограничивать минимальную сумму): -50

Введите максимальную сумму транзакции (Enter — не ограничивать максимальную сумму): 150

Как и для других фильтров, здесь можно не вводить минимальную или максимальную сумму. В этом поиск не будет ограничивать меньшую или большую сумму транзакции соответственно.

Для поиска по этому фильтру нужно учитывать, что для транзакций типа `Taxable` и `ForeignCurrency` реальная сумма транзакции должна отличаться от прочитанной из файла. В первом случае она ниже на величину процента налога, а во втором — определяется курсом валюты транзакции.

Для повторяющихся транзакций достаточно учитывать только исходную сумму транзакции, прочитанную из файла.

Поиск по комментарию

При выборе пункта 5 можно ввести фильтр для поиска по комментарию:

Введите комментарий или его часть (Enter — для поиска всех комментариев): book

Этот фильтр необходим только для операций, которые поддерживают комментирование, то есть для транзакций типа `Commentable`.

Поиск по комментарию должен осуществляться методом подстрочного поиска, где введённое пользователем ключевое слово сравнивается с любой частью комментария. Регистр при этом не учитывается. Например, если пользователь вводит `he`, программа должна вернуть все транзакции, где в комментарии есть такие подстроки, например `HelloWorld`, `He works hard`, `theater` или `Ekatherina`.

Группировка транзакций

В этом меню пользователь может выбрать поле, по которому будут группироваться отфильтрованные транзакции:

Выберите опцию группировки транзакции:

- 0 — вернуться назад (без группировки);
- 1 — группировать по месяцам;
- 2 — группировать по годам;
- 3 — группировать по дню недели;
- 4 — группировать по категории;
- 5 — считать доходы и расходы;
- 6 — группировать по типу счёта;
- 7 — группировать по ID пользователя.

Введите нужную опцию и нажмите Enter.

Если пользователь выберет опцию 0, то вернётся в главное меню, а транзакции не сгруппируются. Опция 5 позволяет сгруппировать транзакции по признаку «Доход» (сумма транзакции больше нуля) или «Расход» (сумма транзакции меньше нуля). Опция 6 позволяет сгруппировать транзакции по типу счёта. Он указан в файле со счетами: 0 — текущий, 1 — сберегательный, 2 — кредитный счёт. Опция 7 позволяет сгруппировать транзакции по ID пользователя, который тоже указан в файле со счетами.

Функция агрегации

В этом меню пользователь может выбрать способ агрегации транзакций:

Выберите способ агрегации транзакций:

- 0 — подсчёт суммы;
- 1 — подсчёт среднего значения;
- 2 — подсчёт количества.

Введите нужную опцию и нажмите Enter.

После выбора способа агрегации пользователь возвращается в главное меню, где может рассчитать аналитику по транзакциям и сохранить результат расчёта.

Расчёт аналитики

Четвёртый пункт главного меню предназначен для расчёта аналитики для транзакций, которые загружены из файла и удовлетворяют выбранным критериям фильтрации.

Пользователь может сразу выбрать этот пункт, если хочет получить аналитику по всем транзакциям, а в качестве агрегирующей функции использовать суммирование. Разберём несколько примеров работы программы с разными способами группировки и функциями агрегации.

Пусть в исходном файле с транзакциями содержатся следующие данные:

```
10001,2,2020-01-01T12:00:00,Food,-25.50,Regular,  
10001,4,2020-01-15T08:30:00,Transport,-15.00,Regular,  
10001,6,2020-03-22T19:00:00,Entertainment,-45.00,ForeignCurrency,85  
10002,8,2020-04-15T14:15:00,Salary,5000.00,Taxable,0.20  
10002,10,2021-05-10T11:00:00,Salary,100.00,Taxable,0.10  
10003,12,2021-05-18T16:20:00,Entertainment,-75.99,ForeignCurrency,85  
10003,14,2022-03-05T07:45:00,Food,-12.50,Regular,
```

Напомним, какая информация содержится в полях по порядку:

- 1) ID счёта;
- 2) ID транзакции;
- 3) дата транзакции;
- 4) категория;
- 5) сумма транзакции;
- 6) тип транзакции и дополнительные данные.

Чтобы объяснить, как работает группировка и агрегация, будем считать, что пользователь работает со всеми доступными транзакциями, не используя фильтрацию.

Представьте, что выбрана группировка по годам, а в качестве агрегирующей функции — сумма. Тогда программа должна вернуть сумму всех транзакций по каждому году. При этом нужно учесть, что из суммы транзакций с ID 8 и 10 необходимо вычесть налог. В итоге для транзакции с ID 8 получим $5000 \text{ руб.} - 20\% = 4000 \text{ руб.}$, для ID = 10: $100 \text{ руб.} - 10\% = 90 \text{ руб.}$

В транзакциях с ID 6 и 12 сумма указана в иностранной валюте. Чтобы перевести её в базовую, сумму нужно умножить на переводной курс. В этом данном случае — 85 для обеих транзакций. В итоге получим: $-45 \times 85 = -3825$ руб. Для ID = 6 и $-75,99 \times 85 = -6459,15$ руб. Округление необходимо производить до двух знаков по стандартным правилам округления:

- если десятичная часть числа (после запятой) больше или равна 0,5, число округляется в большую сторону: $4,8 \approx 5$;
- если десятичная часть числа меньше 0,5, число округляется в меньшую сторону: $4,3 \approx 4$.

Объединяем таким образом транзакции по годам и получаем итог по каждому году:

2020: $-25,50 + (-15,00) + (-3825,00) + 4000,00 = 134,50$ руб.

2021: $90,00 + (-6459,15) = -6369,15$ руб.

2022: $-12,50$ руб.

При выборе количественной функции мы получим число транзакций в каждый год:

2020: 4

2021: 2

2022: 1

При агрегации путём расчёта среднего значения — среднюю сумму чека по каждому году (округлять эти данные не нужно):

2020: $134,50 / 4 = 33,625$

2021: $-6369,15 / 2 = -3184,575$

2022: $-12,5 / 1 = -12,5$

Для примера разберём функции агрегации при группировке по категориям.

Суммирование

Salary: $4000 + 90 = 4090$ руб.

Entertainment: $-3825 + (-6459,15) = -10284,15$ руб.

Transport: $-15,00$ руб.

Food: $-25,50 + (-12,50) = -38,00$ руб.

Количество

Salary: 2

Entertainment: 2

Transport: 1

Food: 2

Среднее

Salary: $4090 / 2 = 2045$

Entertainment: $-10284,15 / 2 = -5142,075$

Transport: $-15,00 / 1 = -15,00$

Food: $-38,00 / 2 = -19,00$

Если выбран вариант «без группировки», программа выведет показатели отдельно по каждой транзакции. В примерах ниже первое число — это ID транзакции.

Суммирование

```
2: -25,50 руб.  
4: -15,00 руб.  
6: -6459,15 руб.  
8: 4000,00 руб.  
10: 90,00 руб.  
12: -3825 руб.  
14: -12,50 руб.
```

Количество

```
2: 1  
4: 1  
6: 1  
8: 1  
10: 1  
12: 1  
14: 1
```

Среднее:

```
2: -25,50 руб.  
4: -15,00 руб.  
6: -6459,15 руб.  
8: 4000,00 руб.  
10: 90,00 руб.  
12: -3825 руб.  
14: -12,50 руб.
```

Благодаря примерам вы можете детальнее разобраться в логике группировки и агрегирования данных по транзакциям. Некоторые сложности могут возникнуть, когда вы будете группировать по типу счёта или по ID пользователя. В этом случае придётся придумать, как связать счета и транзакции.

Учёт повторяющихся транзакций в аналитике может быть сложным для начинающих разработчиков, поэтому реализация такого функционала остаётся на ваше усмотрение. Для успешной сдачи проекта достаточно учитывать в расчёте одно выполнение транзакции.

Вывод аналитики

Выбирая этот пункт меню, пользователь должен увидеть результат расчёта аналитики по отфильтрованным транзакциям, сгруппированных по выбранному полю и выбранной функции агрегации.

Вывод может включать:

- дату и время расчёта;
- выбранные фильтры;

- поле группировки;
- функцию агрегации.

Пример вывода аналитики в консоль:

```
=====
Дата: 2024-10-09 17:11:41
Имя: 'группировка по категории (подсчёт суммы)'
Фильтр: , начальная дата — 2020-01-01, конечная дата — 2021-05-11,
минимальная сумма — -40, максимальная сумма — 1000
-----
Аналитика:
Salary: 90.0000
Transport: -15.00
Food: -25.50
```

Вы можете выбрать любой удобный формат для вывода аналитики.

Сохранение аналитики

После расчёта аналитики по транзакциям пользователь может сохранить результат в файл. Если он предварительно не выбрал пункт меню для расчёта аналитики, должно появляться предупреждение, что аналитика ещё не рассчитана:

Необходимо сначала рассчитать аналитику

Имя файла, в который происходит запись, конфигурируется при запуске проекта. Рекомендуемый формат записи результата — JSON. Сохранённый результат предыдущего расчёта выглядит так:

```
{
  "date" : [ 2024, 10, 9, 17, 11, 41, 558770400 ],
  "groupOption" : "группировка по категории",
  "aggregateOption" : "подсчёт суммы",
  "filter" : ", начальная дата — 2020-01-01, конечная дата — 2021-05-11,
минимальная сумма — -40, максимальная сумма — 1000",
  "data" : {
    "Salary" : 90.0000,
    "Transport" : -15.00,
    "Food" : -25.50
  }
}
```

Сохраняются все те же данные: дата, поле группировки, функция агрегирования, фильтры, выбранные пользователем, и данные расчёта. Вы можете выбрать имена и форматы полей JSON-файла самостоятельно в соответствии с вашим представлением о структуре и содержании данных.

Рекомендации по структуре проекта

Ваш проект должен следовать концепции трёхзвенной архитектуры с чётким делением на уровни представления, бизнес-логики и доступа к данным. Такой подход обеспечивает масштабируемость, удобство сопровождения и модульность системы.

Уровень представления (Presentation Layer)

Отвечает за взаимодействие с пользователем, то есть собирает входные данные от пользователей и отображает результаты. В этом проекте взаимодействовать с пользователями позволяет консоль. Чтобы вывести всю стандартную информацию (например, результаты обработки данных, сообщения о выполнении операций, запросы к пользователю), применяем `System.out.println()`:

```
System.out.println("Введите значение:");  
System.out.println("Результат: " + результат);
```

Для вывода сообщений об ошибках, исключениях или других предупреждениях, которые могут потребовать внимания при отладке и тестировании, лучше использовать `System.err.println()`:

```
System.err.println("Ошибка: не удалось прочитать файл.");  
System.err.println("Исключение: " + exception.getMessage());
```

Часто реализация работы с консолью является трудоёмкой, поэтому в шаблоне проекта мы подготовили готовые классы, реализующие этот функционал. Внимательно ознакомьтесь с реализацией классов `AbstractMenuController`, `MainMenuController` и `SearchMenuController` в пакете `controller`.

Реализуйте собственные классы:

- `AggregateMenuController` для выбора способа агрегации;
- `GroupMenuController` для выбора поля группировки транзакций.

Обратите внимание на классы-перечисления `enum` в пакете `controller.option`.

Реализацию классов `AggregateOption`, `GroupOption` необходимо исправить.

Готовые классы `MainMenuOption` и `SearchOption` должны помочь вам понять логику классов-перечислений для меню.

В пакете `controller.dto` находится реализация класса для хранения параметров фильтра для транзакций `TransactionFilterDto`. Вам необходимо добавить в этот класс все нужные поля и методы для получения предикатов по этим полям.

Внимательно изучите файлы в пакете `controller`. Методы и классы, которые нужно реализовать, помечены такими комментариями:

```
// TODO: реализуйте метод...
```

IntelliJ IDEA обладает встроенным функционалом поиска подобных комментариев. Подробнее об этом можно узнать [в справочнике](#).

Используйте это окно, чтобы отслеживать функционал, который нужно реализовать. Когда он будет готов, удалите соответствующий комментарий `TODO`, а в конце убедитесь, что подобных комментариев не осталось.

Уровень доступа к данным (Data Access Layer)

Отвечает за доступ к данным и взаимодействие с хранилищем данных, а также реализует операции чтения и записи данных в источник данных.

В качестве источника данных в нашем проекте служат текстовые файлы: из них читаются исходные данные для анализа и в них записываются результаты расчётов.

В шаблоне проекта подготовлены необходимые интерфейсы и абстрактные классы для данных — пакет `data.model`. В пакете `data.repository` содержатся интерфейсы сервисов для доступа к данным (репозитории) `AccountRepository`, `AnalyticRepository`, `TransactionRepository`. Вам нужно самостоятельно реализовать эти интерфейсы и добавить необходимый функционал в абстрактные классы.

Уровень бизнес-логики (Business Logic Layer)

Отвечает за обработку данных и бизнес-логику приложения, исполняет основные правила бизнес-логики, которые применяются к данным.

В шаблоне проекта в пакете `service` находится интерфейс `TransactionService`, вам необходимо имплементировать его. Единственный метод интерфейса принимает на вход:

- фильтр для транзакций;
- опцию группировки и агрегации.

Возвращает аналитику по транзакциям, прочитанным из файла.

При реализации интерфейсов важно помнить, что каждый слой должен отвечать за самостоятельный аспект обработки данных: представление, бизнес-логику или доступ к данным.

Необходимо соблюдать принцип, при котором каждый слой взаимодействует только со своим непосредственным соседом. Исключение — это уровень представления, который не обращается напрямую к уровню данных, а только через уровень бизнес-логики.

Нужно реализовать отдельные исключения для каждой возможной бизнес-ошибки. Для этого в шаблон добавлен пакет `exception`.

Рекомендации и советы

- Убедитесь, что вы чётко понимаете, как работает трёхзвенная архитектура и какие обязанности имеет каждый её слой.

- Перед тем как начать кодировать, составьте план приложения. Нарисуйте диаграмму, которая покажет, как будет организована архитектура и взаимодействие различных компонент.
- Соблюдайте принципы инкапсуляции. Каждый слой должен иметь чёткие границы и скрывать свою внутреннюю реализацию.
- Убедитесь, что каждый слой выполняет только свои функции: представление, бизнес-логику или доступ к данным.
- Вы можете использовать библиотеку `Lombok`, чтобы упростить и ускорить процесса разработки. `Lombok` предоставляет аннотации, которые автоматически генерируют стандартный код Java: геттеры, сеттеры, методы `equals()`, `hashCode()`, `toString()` и многое другое. Это позволяет вам сосредоточиться на бизнес-логике и избавляет от рутины написания шаблонного кода.
- В приложении предусмотрено множество способов группировки полей транзакций и применения агрегирующих функций. Написание отдельного кода для каждого варианта может привести к путанице. Даже преподавателю сложно понять такой код. Чтобы избежать этого, попробуйте использовать знания о дженериках. Сконцентрируйтесь на создании отдельных функций для группировки и «коллекторов» для агрегации данных. Тогда вам нужно лишь объединить эти функции, коллекторы и исходные данные.
- В шаблон проекта включён файл `build.gradle`, при помощи которого Gradle управляет зависимостями и автоматизирует сборку проекта. Это позволит вам эффективно управлять зависимостями, компилировать проект и создавать сборки с минимальными усилиями.
- При открытии проекта с файлом `build.gradle` IntelliJ IDEA может автоматически определить, что этим проектом управляет Gradle, и предложить выполнить импорт. Вам стоит согласиться на импорт, чтобы интегрированная среда разработки настроила проект с учётом используемых зависимостей и конфигураций.
- Во время работы над проектом вам могут понадобиться некоторые зависимости, например для библиотеки `Lombok` или для библиотек, позволяющих преобразовывать экземпляры классов в текстовый JSON. В центральном репозитории [Apache Maven](#) вы найдёте актуальные версии этих библиотек и нужные объявления зависимостей для Gradle.
- Используйте системы управления версиями (Git), чтобы отслеживать изменения в коде и упрощать работу в команде. После клонирования проекта из репозитория создайте новую ветку от ветки `master` и ведите в ней разработку. Так вам будет проще подготовить пул-реквест перед сдачей проекта.

- Пишите понятный и поддерживаемый код. Следуйте стандартам кодирования, используйте осмысленные имена для переменных и методов, форматируйте код, чтобы улучшить читаемость.
- Заботьтесь о качественной обработке ошибок, чтобы приложение не падало при возникновении нештатных ситуаций.
- В шаблоне проекта находятся файлы с транзакциями `transaction.txt` и счетами `accounts.txt`. Эти файлы содержат данные, примеры расчёта аналитики для которых приведены в разделе «Расчёт аналитики». После реализации функционала приложения сравните полученные результаты с примерами, рассмотренными в этом разделе.

Формат сдачи

Проект сдаётся в виде ссылки на Merge Request в репозитории GitLab.

Критерии оценивания

Чтобы успешно сдать проект, нужно выполнить следующие требования:

- проект компилируется и запускается без ошибок;
- все заявленные функции реализованы на должном уровне и соответствуют требованиям проекта;
- в готовом проекте нет комментариев вида *TODO*, *FixMe*, а также закоментированного кода;
- уровни приложения правильно разделены: представление (Presentation), бизнес-логика (Business Logic), доступ к данным (Data Access);
- границы между слоями не нарушаются, например слой «Представления» напрямую обращается к слою «Доступ к данным»;
- уровни не должны быть избыточно связаны друг с другом;
- модульность и расширяемость системы повышается при помощи интерфейсов и абстракций;
- реализованы механизмы для обработки нештатных ситуаций, а для обработки бизнес-ошибок созданы собственные исключения;
- использована система контроля версий для отслеживания изменений в проекте, коммиты содержат понятные сообщения с описанием, а ветки имеют ясную структуру.

Желаем успехов в работе над проектом!